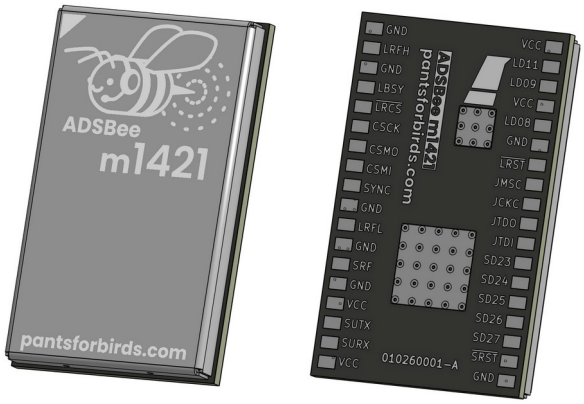
# ADSBee m1421

Ultra Low-Power Dual Band ADS-B Receiver Module




## Applications

- Aircraft detect and avoid for UAS
- ADS-B In for low-power systems

## Quick Specs

| | |
|---|---|
| Power Draw | 19mA @ 1.8-3.3V |
| Sensitivity | Mode S: -90dBm to 0dBm<br>UAT: -90dBm to 0dBm |
| Simultaneous Aircraft Tracks Supported | ≤400 |
| Weight | 1.85g |

## Features

- 1090MHz Mode S and ADS-B packet decoding.
- Tunable sub-GHz transceiver for UAT / additional protocols.
- Multiple output formats over UART: Raw Frames / ADSBee CSV / MAVLINK1 / MAVLINK2 / Mode S Beast / GDL90 / Aircraft JSON.
- Firmware updates over UART or JTAG.

# Contents


Features....1
Applications....1
Quick Specs....1
Revision History....4
Safety Notice....5
Overview....6
1 Pin Functions....7
1.1 SYNC Pin Functionality....8
2 Device Form Factor....9
2.1 Module Dimensions....9
2.2 Recommended Footprint....10
3 AT Commands....11
3.1 AT+BAUD_RATE....11
3.1.1 Set the Console Baud Rate....11
3.1.2 Query the Console Baud Rate....11
3.2 AT+BOOTLOADER_PIN....12
3.2.1 Enable or Disable the Bootloader Pin....12
3.2.2 Query the Bootloader Pin....12
3.3 AT+CONFIG_HASH....13
3.4 AT+DEVICE_INFO....14
3.5 AT+FILTER_ALTITUDE_FT....15
3.5.1 Set the Altitude Filter....15
3.5.2 Query the Altitude Filter....15
3.6 AT+FILTER_DISTANCE_NM....15
3.6.1 Set the Distance Filter....15
3.6.2 Query the Distance Filter....16
3.7 AT+LOG_LEVEL....16
3.7.1 Set the Log Level....16
3.7.2 Query the Log Level....16
3.8 AT+MAVLINK_ID....17
3.8.1 Set the MAVLINK System ID and Component ID....17
3.8.2 Query the MAVLINK System ID and Component ID....17

3.9 AT+PROTOCOL_OUT....19
3.9.1 Set the Reporting Protocol for a Serial Interface....19
3.9.2 Query the Reporting Protocol for All Serial Interfaces....19
3.10 AT+R1090_GAIN....19
3.10.1 Set the 1090 MHz Gain....19
3.10.2 Query the 1090 MHz Gain....19
3.11 AT+R1090_PREAMBLE....20
3.11.1 Set the Preamble Mode....20
3.11.2 Query the Preamble Mode....20
3.12 AT+R1090_RX_BOOST....21
3.12.1 Set the 1090 MHz RX Boost Level....21
3.12.2 Query the 1090 MHz RX Boost Level....21
3.13 AT+REBOOT....22
3.14 AT+RX_ENABLE....23
3.14.1 Enable or Disable the Receiver(s)....23
3.14.2 Query Whether the Receiver(s) Are Enabled....23
3.15 AT+RX_POSITION....24
3.15.1 Query Receiver Position....24
3.15.2 Set Receiver Position....24
3.16 AT+RX_STATS....26
3.17 AT+SETTINGS....27
3.17.1 Query Current Settings....27
3.17.2 Dump Settings in AT Format....27
3.17.3 Load Settings from Nonvolatile Memory....27
3.17.4 Save Settings to Nonvolatile Memory....27
3.17.5 Reset Settings to Factory Defaults....27
3.18 AT+SUBG_MODE....28
3.18.1 Set the Sub-GHz Receiver Mode....28
3.18.2 Query the Sub-GHz Receiver Mode....28
3.19 AT+WATCHDOG....29
3.19.1 Configure the Watchdog Timer....29
3.19.2 Query the Watchdog Timer....29
3.19.3 Test the Watchdog Timer....29
4 Reporting Protocols....30
4.1 CSBee....31

4.1.1 CSBee Protocol Version History....31
4.1.2 Mode S Aircraft Message....32
4.1.3 UAT Aircraft Message....39
4.1.4 Statistics Message....41
4.2 GDL90....42
4.3 MAVLINK....43
4.3.1 MAVLINK_HEARTBEAT （ Message ID 0 ） Packet Definition....43
4.3.2 MAVLINK ADSB_VEHICLE (Message ID 246) Packet Definition....43
4.3.3 MAVLINK_REQUEST_DATA_STREAM (Message ID 66) Packet Definition....44
4.3.4 MAVLINK MESSAGE_INTERVAL (Message ID 244) Packet Definition....44
4.4 Mode S Beast....45
4.4.1 Beast: Mode S Frames....45
4.4.2 Beast: UAT Frames....45
4.5 Raw Packets....46
4.5.1 Raw Mode S Frames....46
4.5.2 Raw UAT ADSB Frames....46
4.5.3 Raw UAT Uplink Frames....47
4.6 Aircraft JSON....48
4.6.1 Message Format....48
4.6.2 JSON Fields....49

# Revision History

20260516

- Initial release.

20260606

- Add detailed protocol descriptions.
- Revise power estimate.
- Documented the serial command interface and reporting protocols. Renamed the 1090 MHz commands to the R1090_ prefix and added RX_STATS.

20260818

- Added AT+SUBG_MODE for selecting the Sub-GHz receiver mode (UAT_RX / UAT_RX_NO_UPLINK, disables UAT uplink reception). AT+RX_ENABLE now powers the Sub-GHz receiver down when disabled.

20261016

- AT+R1090_PREAMBLE now has two modes: MODE_S (new default, every downlink format) and DF17 (DF17 frames only). AT+R1090_GAIN AUTO is automatic gain control in DF17 and gain step 13 in MODE_S. Saved settings reset to factory defaults when upgrading to this firmware.
- Rebuilt the table of contents to match the section numbers and pages. Firmware references updated to 0.3.11.
- Added AT+CONFIG_HASH for comparing the settings, CCFG and firmware image of a module with a reference module.
- Corrected the note in the MAVLINK MESSAGE_INTERVAL packet definition: interval_us is always 1000 us.
- Added AT+FILTER_DISTANCE_NM for leaving aircraft beyond a set distance (in nautical miles) from the receiver out of aircraft reports; raw packets are never filtered. Aircraft JSON now includes r_dst and r_dir (distance and bearing from the receiver).
- Added AT+FILTER_ALTITUDE_FT for leaving aircraft outside a set altitude band (above, below, between two altitudes, or near the receiver altitude) out of aircraft reports; raw packets are never filtered.
- Sensitivity is -90dBm to 0dBm for Mode S and UAT.

## Safety Notice

We work hard to make ADSBee products performant and reliable, but not all aircraft broadcast their location, messages can be lost or corrupted, and hardware or software can fail. ADSBee cannot guarantee detection or accurate reporting of all traffic. Users are responsible for evaluating its suitability for their application and for the safe operation of any system it is integrated into. See our terms of sale (pantsforbirds.com/terms-of-sale) and license (github.com/CoolNamesAllTaken/adsbee) for full terms.

# Overview

ADSBee m1421 is the world's lowest power and most cost-effective dual-band ADS-B receiver. The name comes from what's under the hood: a TI CC13**14** low power RF MCU and a Semtech LR20**21** radio. These two ICs combine the best of two worlds: the LR2021's best in class sensitivity for high baud rate OOK RF protocols, plus the CC1314's low-power compute and high sensitivity sub-GHz RF reception.

ADSBee m1421 is built with open-source firmware from Pants for Birds LLC's ADSBee project, and includes onboard demodulation and decoding for Mode S and UAT messages, with ADS-R, TIS-B, and FIS-B (including NOTAMs and weather). An onboard aircraft dictionary contains all relevant information for up to 400 aircraft targets, and reports target information via UART using a wide range of available protocols, including RAW, CSBee, MAVLINK1, MAVLINK2, GDL90, and Mode S Beast.

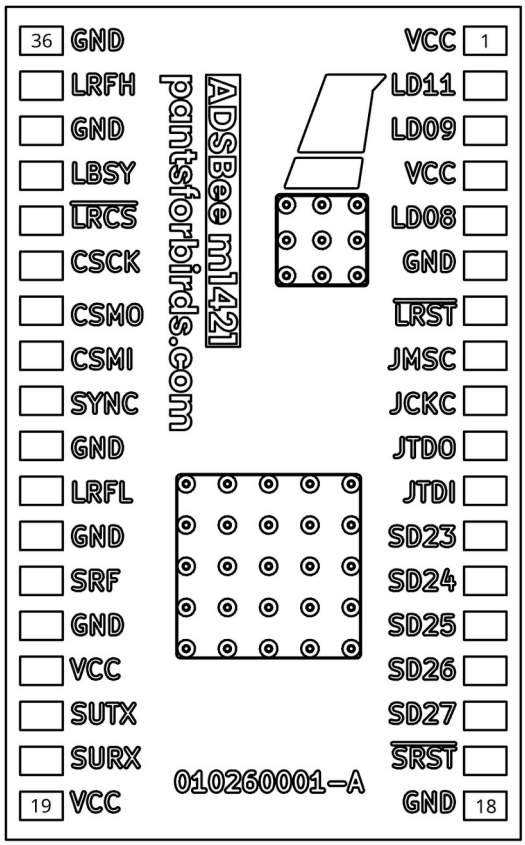
Customers who want direct control of the onboard LR2021's 2.4GHz radio (e.g. for LoRa or mesh radio applications) can suspend the CC1314 using the SYNC line and configure the LR2021 directly over SPI. All LR2021 SPI pins, plus three GPIOs, are provided for direct interfacing with the radio (interrupts, etc).

# 1 Pin Functions

| Pin # | Pin Name | Pin Function | Pin # | Pin Name | Pin Function |
|---|---|---|---|---|---|
| 1 | VCC | Power supply (1.8V – 3.3V). | 19 | VCC | Power supply (1.8V – 3.3V). |
| 2 | LD11* | LR2021 DIO 11. | 20 | SURX | SubGHz MCU UART RX. |
| 3 | LD09* | LR2021 DIO 12. | 21 | SUTX | SubGHz MCU UART TX. |
| 4 | VCC | Power supply (1.8V – 3.3V). | 22 | VCC | Power supply (1.8V – 3.3V). |
| 5 | LD08* | LR2021 DIO 8. | 23 | GND | Ground. |
| 6 | GND | Ground. | 24 | SRF | SubGHz RF connection. |
| 7 | ~LRST* | LR2021 Reset (active low). | 25 | GND | Ground. |
| 8 | JMSC | SubGHz MCU JTAG TMSC. | 26 | LRFL | Mode S RF connection. |
| 9 | JCKC | SubGHz MCU JTAG TCKC. | 27 | GND | Ground. |
| 10 | JTDO | SubGHz MCU JTAG TDO. | 28 | SYNC* | SubGHz sync (drive high to suspend SubGHz MCU). |
| 11 | JTDI | SubGHz MCU JTAG TDI. | 29 | CSMI* | LR2021 SPI MISO. |
| 12 | SD23 | SubGHz MCU DIO 23. | 30 | CSMO* | LR2021 SPI MOSI. |
| 13 | SD24 | SubGHz MCU DIO 24. | 31 | CSCK* | LR2021 SPI SCK. |
| 14 | SD25 | SubGHz MCU DIO 25. | 32 | ~LRCS* | LR2021 SPI CS. |
| 15 | SD26 | SubGHz MCU DIO 26. | 33 | LBSY* | LR2021 busy. |
| 16 | SD27 | SubGHz MCU DIO 27. | 34 | GND | Ground. |
| 17 | ~SRST | SubGHz MCU Reset (active low). | 35 | LRFH* | LR2021 2.4GHz RF connection. |
| 18 | GND | Ground. | 36 | GND | Ground. |
| MP | | Thermal pad for CC1314 and LR2021. | | | |

*= Only required for direct control of LR2021. Leave unconnected for normal applications.

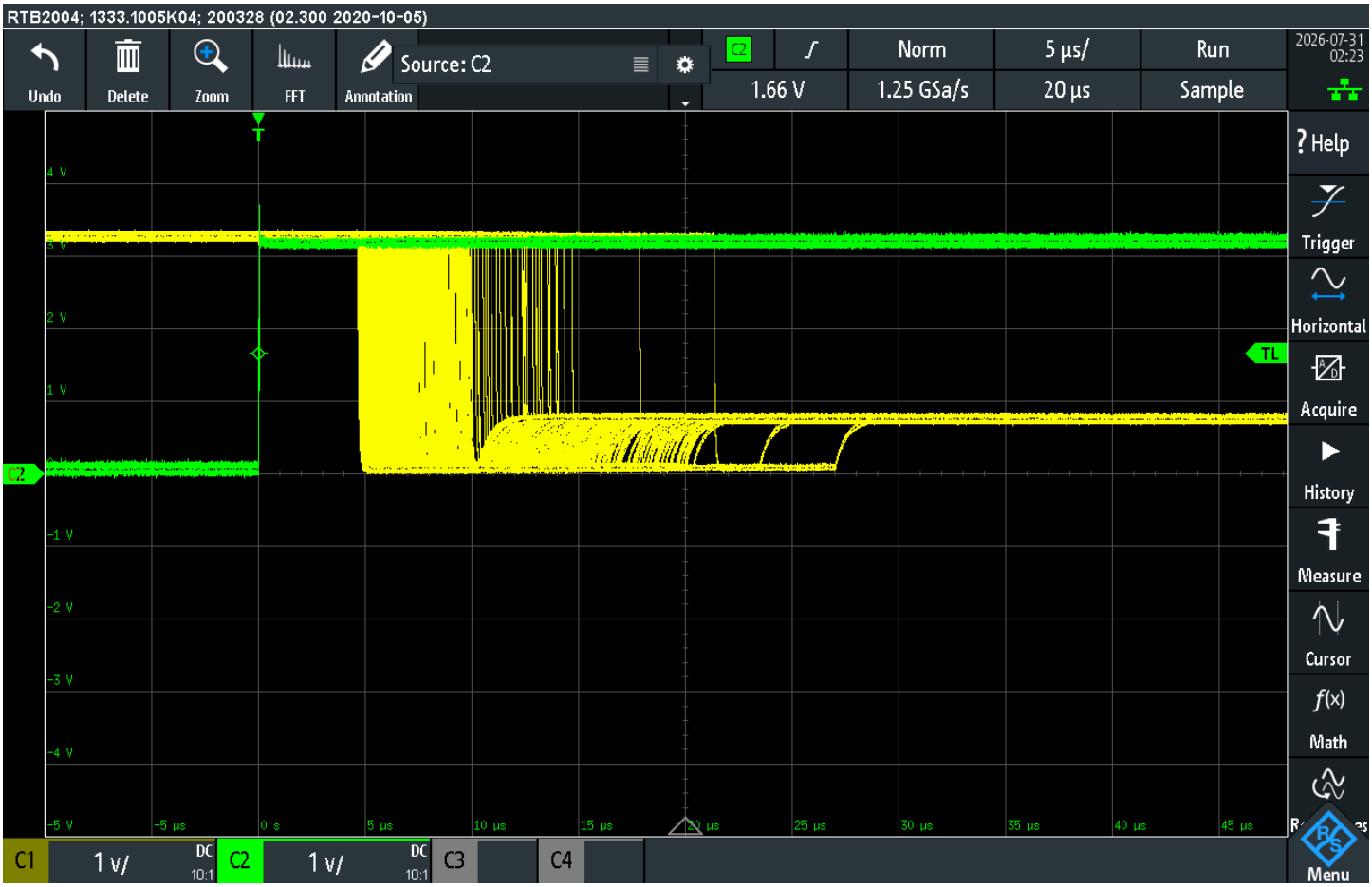
MODULE BACK SIDE
(PINS ARE REVERSED)



## 1.1 SYNC Pin Functionality

The SYNC pin can be driven high to put the CC1314R10 into a low-power sleep state, with all LR2021 control lines tri-stated in order to allow external control of the LR2021.

The green trace below shows the SYNC line going high, and the yellow trace shows the LRST (LR2021 RESET) net going to tri-state, in a repeated test with trace persistence turned to maximum. To be safe, an external MCU may assume that the LR2021is freed 40us after asserting the SYNC line high, or after the LRST line is tri-stated, whichever occurs first.

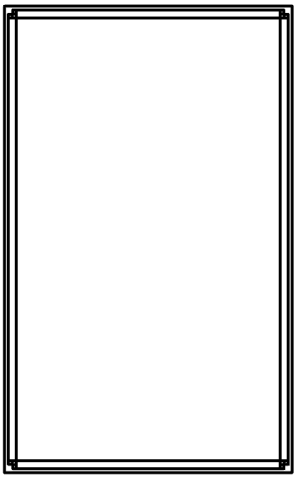



**NOTE:** If you put ADSBee m1421 to sleep for longer than the watchdog timer interval (10s by default), the watchdog will fire during sleep and reset the module. If this happens while SYNC is high, ADSBee m1421 will enter the bootloader. For applications where ADSBee m1421 is slep longer than the watchdog interval, it is recommended to disable the watchdog timer with `AT+WATCHDOG=0`

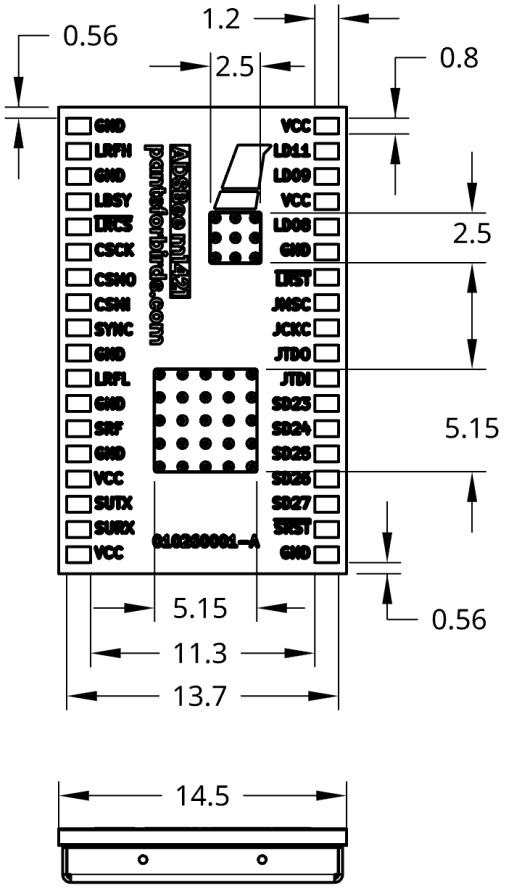
# 2 Device Form Factor

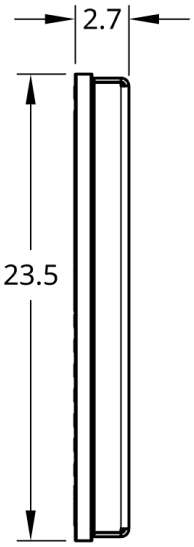
## 2.1 Module Dimensions

TOP VIEW



BOTTOM VIEW







NOTE:
Dimensions in mm.

All vias shown on module
are epoxy filled and capped.

## 2.2 Recommended Footprint

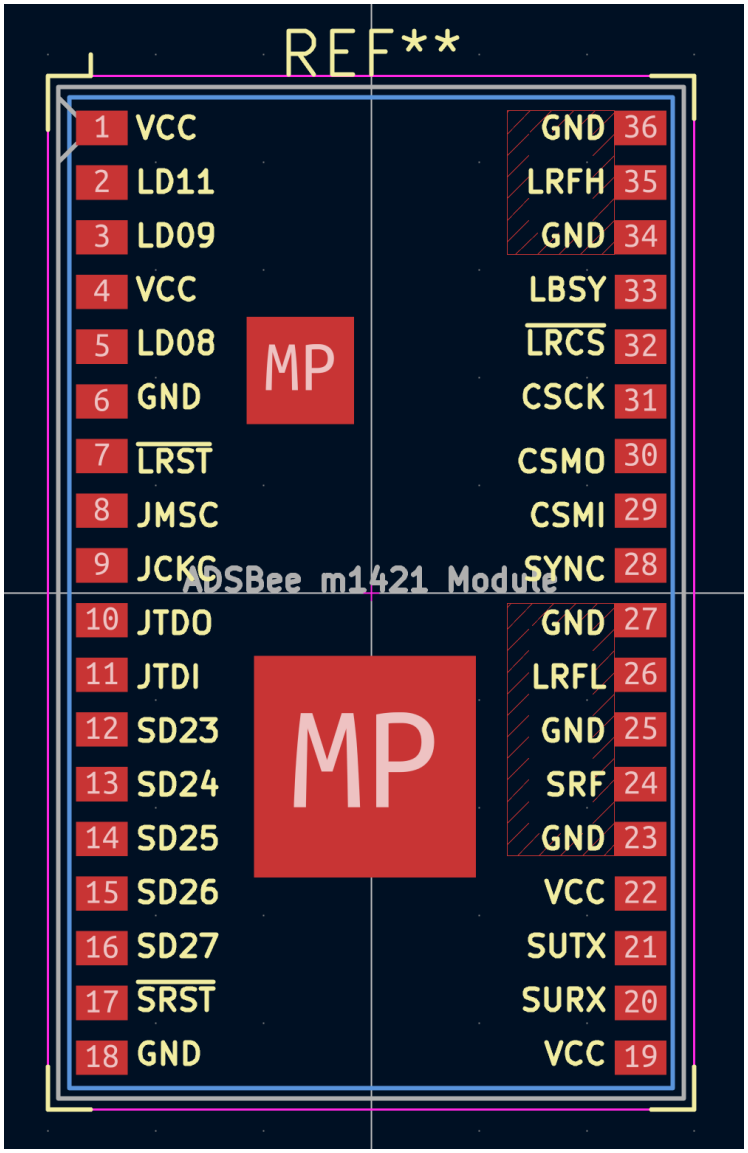
A KiCAD footprint is available at: https://github.com/CoolNamesAllTaken/kicad-libs/blob/main/lib_fp/Custom_Module.pretty/PantsForBirds_ADSBee_m1421_Module.kicad_mod

A screenshot of the footprint is included below. Note the top copper keepouts (hatched red areas) where impedance controlled transmission lines exist on the surface of the module.

LRFH, LRFL, and SRF are coplanar waveguides controlled to 50 Ohms. They can be broken out as microstrip or coplanar waveguide, depending on designer preference. Inserting filled and capped vias directly under the GND pads on the right side of the module is recommended for minimal impedance discontinuity.

LR2021 and CC1314 IC mounting pads are brought through the module using thermal vias. It is recommended to connect both pads labelled “MP” to a ground plane using filled and capped thermal vias.

All exposed copper pads shown in red should have paste.



# 3 AT Commands

ADSBee m1421 is configured over its Console UART using ASCII AT commands. Each command begins with the prefix AT+ and ends with a carriage return. Most commands provide a query form (AT+COMMAND?) that reports the current value and a set form (AT+COMMAND=<args>) that changes it. Changes take effect immediately; write them to nonvolatile memory with AT+SETTINGS=SAVE so they persist across reboots.

## 3.1 AT+BAUD_RATE

Changes the Console UART baud rate at runtime. The device acknowledges with OK at the old baud rate and then switches, so the host should reopen its serial port at the new rate after receiving the OK. The baud rate is not saved to nonvolatile settings: any reboot or power cycle returns the console to 115200 baud.

A baud rate of 921600 or above is recommended for best performance. Significant bottlenecks can occur if using verbose reporting protocols at the default baud rate of 115200.

### 3.1.1 Set the Console Baud Rate

```
AT+BAUD_RATE=CONSOLE,<baud>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| iface | Serial interface to configure. | CONSOLE |
| baud | New console baud rate. | 115200, 230400, 460800, 921600, 1000000 |

### 3.1.2 Query the Console Baud Rate

```
AT+BAUD_RATE?
BAUD_RATE=CONSOLE,115200
```

## 3.2 AT+BOOTLOADER_PIN

Enables or disables the bootloader backdoor in the CC1314 CCFG (customer configuration) flash. With the backdoor enabled, which is the factory default, holding SYNC high while ~SRST is pulsed low starts the CC1314 ROM UART bootloader on pins DIO_2/DIO_3. The ADSBee 1421 Programmer and other host tools use this to reflash the module. The setting takes effect at the next reset and persists across power cycles and AT+SETTINGS=RESET. Every write is read back and verified; a write that fails prints a CCFG WRITE FAILED error banner with the read-back value and the recovery steps, and returns ERROR.

**Warning:** With the backdoor disabled, SYNC no longer enters the bootloader, so the ADSBee 1421 Programmer and other SYNC-based tools cannot reflash the module. The Programmer (0.3.11 and later) still bridges the console, without checking the image, so AT+BOOTLOADER_PIN=1,DEADBEE can be sent through it. The only ways to enable it again are AT+BOOTLOADER_PIN=1,DEADBEE sent to the running firmware, or a JTAG flash. Flashing a complete firmware image writes that image's CCFG, which has the backdoor enabled in every release.

### 3.2.1 Enable or Disable the Bootloader Pin

```
AT+BOOTLOADER_PIN=<enabled>,<password>[,DRYRUN]
BL_CONFIG 0xC5FF0500 -> 0xC5FF05C5: erase and reprogram the CCFG sector
CCFG verified: BL_CONFIG 0xC5FF05C5, CCFG CRC32 0x66E9858D.
Bootloader backdoor enabled; takes effect at the next reset.
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| enabled | 1: SYNC high while ~SRST is pulsed low starts the ROM UART bootloader. 0: SYNC is ignored at reset. | 0, 1 |
| password | Required for every set form, DRYRUN included, so the CCFG is only written on purpose. A missing or wrong password returns ERROR and leaves the CCFG untouched. Queries need no password. | DEADBEE |
| DRYRUN | Optional. Prints the current and new BL_CONFIG values and the write method, and does not write flash. | DRYRUN |

### 3.2.2 Query the Bootloader Pin

```
AT+BOOTLOADER_PIN?
BOOTLOADER_PIN=1
BL_CONFIG=0xC5FF05C5: ROM bootloader enabled, backdoor enabled on DIO_5 (active high)
```

## 3.3 AT+CONFIG_HASH

Prints hashes of the settings, the CCFG (customer configuration) and the firmware image, so a module can be compared with a reference module in one line. Two modules with the same firmware and the same settings print the same values. Every hash is a CRC-32.

```
AT+CONFIG_HASH?
CONFIG_HASH=0x5C1D0A37
SETTINGS_FLEET=0x877DBEC3 (settings version 6, 66 Bytes)
SETTINGS_FULL=0x877DBEC3
CCFG_CC1314=0x66E9858D
FIRMWARE=0x1B2E9F40
FIRMWARE_CC1314=0x8A40C3D2 (566424 Bytes)
```

| Line | Description |
|---|---|
| CONFIG_HASH | Hash of SETTINGS_FLEET, CCFG_CC1314 and FIRMWARE. Compare this first. |
| SETTINGS_FLEET | All settings except the receiver position values (the position source still counts). |
| SETTINGS_FULL | All settings. Receiver position values count only for a FIXED position, and the ICAO address only for the ICAO source. |
| CCFG_CC1314 | The CCFG read from flash. AT+BOOTLOADER_PIN=0 changes it. |
| FIRMWARE | Hash of the FIRMWARE_CC1314 line. |
| FIRMWARE_CC1314 | The application image in flash, without the settings and device information storage. |

Settings are hashed as running, saved or not (send AT+SETTINGS=SAVE to keep them). Each release from firmware 0.3.11 on lists its CCFG_CC1314 and FIRMWARE values in adsbee_1421-<version>.hashes.txt.

## 3.4 AT+DEVICE_INFO

Reports manufacturing and firmware information about the module.

```
AT+DEVICE_INFO?
Part Code: ADSBEE-M1421
CC1314R10 Unique ID: 00112233AABBCCDD
CC1314R10 Firmware Version: 0.1.0
CC1314R10 Firmware Build: Release
OTA Key 0:
OTA Key 1:
```

The Firmware Build line reads Debug or Release (firmware 0.3.11 and later). Debug builds add the RF test command AT+TX_CW, which transmits an unmodulated carrier for conducted bench measurements into an attenuator or dummy load. Release builds, including all published releases, don't contain it. To make a Debug build, run ./build.sh -d build_and_flash in firmware/adsbee_1421 (see firmware/README.md).

## 3.5 AT+FILTER_ALTITUDE_FT

Leaves aircraft outside a set altitude band out of aircraft reports: the aircraft reporting protocols (CSBee, MAVLink, GDL90, Aircraft JSON). Raw packets are never filtered: Raw, Beast and MLAT output still carry every packet. The aircraft's altitude is its barometric altitude, or its GNSS altitude when no barometric altitude is known. Aircraft with no known altitude are always kept. Mode 4 compares against the receiver altitude from AT+RX_POSITION (the GNSS altitude for a GNSS fix, otherwise the barometric altitude unless it is 0) and filters nothing while no receiver position is available. With AT+FILTER_DISTANCE_NM also on, an aircraft is reported only if it passes both filters. The filter is off by default. Changes take effect immediately and persist across reboots after AT+SETTINGS=SAVE.

### 3.5.1 Set the Altitude Filter

```
AT+FILTER_ALTITUDE_FT=<mode>,<a_ft>,<b_ft>
OK
AT+FILTER_ALTITUDE_FT=3,1000,18000
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| mode | 0: off. 1: keep aircraft at or above a_ft. 2: keep aircraft at or below a_ft. 3: keep aircraft from a_ft to b_ft. 4: keep aircraft within a_ft above or below the receiver. Bounds are inclusive. Empty keeps the current value. | 0-4 |
| a_ft | Altitude in feet for modes 1-3, or the band half-width in feet for mode 4 (must be greater than 0). Empty keeps the current value. | -2000-100000 |
| b_ft | Upper altitude in feet for mode 3 (at least a_ft); kept but unused in the other modes. Empty keeps the current value. | -2000-100000 |

Default: 0,0,10000 (filter off).

### 3.5.2 Query the Altitude Filter

```
AT+FILTER_ALTITUDE_FT?
FILTER_ALTITUDE_FT=0,0,10000
```

## 3.6 AT+FILTER_DISTANCE_NM

Leaves aircraft that are farther than a set distance from the receiver position out of aircraft reports: the aircraft reporting protocols (CSBee, MAVLink, GDL90, Aircraft JSON). Raw packets are never filtered: Raw, Beast and MLAT output still carry every packet, including those from far aircraft. Aircraft at an unknown distance (no position decoded yet, or no receiver position; see AT+RX_POSITION) are always kept. The filter is off by default. Changes take effect immediately and persist across reboots after AT+SETTINGS=SAVE.

### 3.6.1 Set the Distance Filter

```
AT+FILTER_DISTANCE_NM=<enabled>,<distance_nm>
OK
AT+FILTER_DISTANCE_NM=1,50
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|

| | | |
|---|---|---|
| enabled | 1 turns the filter on, 0 turns it off. Empty keeps the current value. | 0, 1 |
| distance_nm | Aircraft known to be farther than this many nautical miles from the receiver position are left out of aircraft reports. Decimal, 0.1 nm resolution (finer values are rounded). Empty keeps the current value. | 0.1-10000.0 |

Default: 0,100.0 (filter off).

### 3.6.2 Query the Distance Filter

```
AT+FILTER_DISTANCE_NM?
FILTER_DISTANCE_NM=0,100.0
```

## 3.7 AT+LOG_LEVEL

Sets how much diagnostic output the firmware prints to the Console UART.

### 3.7.1 Set the Log Level

```
AT+LOG_LEVEL=<log_level>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| log_level | Console logging verbosity. | SILENT, ERRORS, WARNINGS, INFO |

### 3.7.2 Query the Log Level

```
AT+LOG_LEVEL?
LOG_LEVEL=INFO
```

## 3.8 AT+MAVLINK_ID

MAVLink components have a system ID and a component ID. For more information about system and component IDs, visit https://mavlink.io/en/services/mavlink_id_assignment.html.

### 3.8.1 Set the MAVLINK System ID and Component ID

```
AT+MAVLINK_ID=<system_id>,<component_id>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| system_id | Unique identifier for the MAVLink system (e.g. the aircraft). All components in the system should share the same system_id. | 1-255 |
| component_id | Unique identifier for a component within the MAVLink system (e.g. the ADSBee). Each component should have a unique component_id. | 1-255 |

Default: 156 (MAV_COMP_ID_ADSB).

### 3.8.2 Query the MAVLINK System ID and Component ID

```
AT+MAVLINK_ID?
System ID: 1
Component ID: 156
```

## 3.9 AT+PROTOCOL_OUT

Selects the reporting protocol used to output decoded data on a serial interface. See the Reporting Protocols section for the format of each protocol.

### 3.9.1 Set the Reporting Protocol for a Serial Interface

```
AT+PROTOCOL_OUT=<iface>,<protocol>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| iface | Serial interface to configure. | CONSOLE |
| protocol | Reporting protocol to use on the interface. | NONE, RAW, BEAST, BEAST_NO_UAT, BEAST_NO_UAT_UPLINK, CSBEE, MAVLINK1, MAVLINK2, GDL90, GDL90_NO_UAT_UPLINK, AIRCRAFT_JSON |

### 3.9.2 Query the Reporting Protocol for All Serial Interfaces

```
AT+PROTOCOL_OUT?
PROTOCOL_OUT=CONSOLE,MAVLINK2
```

## 3.10 AT+R1090_GAIN

Sets the gain of the 1090 MHz front-end. AUTO uses the gain that suits the receiver mode: automatic gain control in DF17 mode, fixed gain step 13 in MODE_S mode. A manual step of 1-15 sets a fixed gain in either mode; lower steps reduce sensitivity. Defaults to AUTO.

### 3.10.1 Set the 1090 MHz Gain

```
AT+R1090_GAIN=<gain_step>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| gain_step | 1090 MHz front-end gain step. | 0 or AUTO = mode default (AGC in DF17, step 13 in MODE_S); 1-15 = manual (13 = maximum gain) |

### 3.10.2 Query the 1090 MHz Gain

```
AT+R1090_GAIN?
R1090_GAIN=10
```

## 3.11 AT+R1090_PREAMBLE

Sets the 1090 MHz Mode S receiver mode. MODE_S detects the whole 16-chip preamble at a fixed gain and receives every downlink format, including DF 4/5/11/20/21 (squawk codes, pressure altitude, all-call replies), from weak signals up to strong nearby transmitters. DF17 triggers on the second preamble half plus the DF17 header bits and receives DF17 (ADS-B) frames only. Defaults to MODE_S.

### 3.11.1 Set the Preamble Mode

```
AT+R1090_PREAMBLE=<mode>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| mode | 1090 MHz Mode S receiver mode. | MODE_S, DF17 |

### 3.11.2 Query the Preamble Mode

```
AT+R1090_PREAMBLE?
R1090_PREAMBLE=MODE_S
```

## 3.12 AT+R1090_RX_BOOST

Sets the receive boost level of the LR2021 1090 MHz receive path. Higher boost levels increase receiver sensitivity at the cost of higher current consumption. Defaults to 0 (boost off).

### 3.12.1 Set the 1090 MHz RX Boost Level

```
AT+R1090_RX_BOOST=<boost>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| boost | 1090 MHz RX path boost level. | 0 = off; 1-7 (7 = maximum boost) |

### 3.12.2 Query the 1090 MHz RX Boost Level

```
AT+R1090_RX_BOOST?
R1090_RX_BOOST=0
```

## 3.13 AT+REBOOT

Reboots the device.

```
AT+REBOOT
```

## 3.14 AT+RX_ENABLE

Enables or disables the onboard receivers. The first argument, if present, overrides the others, allowing all receivers to be enabled or disabled at once. Disabling the Sub-GHz receiver powers down the CC1314 RF core; re-enabling it restarts reception in the mode selected by AT+SUBG_MODE.

### 3.14.1 Enable or Disable the Receiver(s)

```
AT+RX_ENABLE=<all>,<1090>,<subg>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| all | Enables or disables all receivers. Overrides the other arguments if present. | 0, 1 |
| 1090 | Enables or disables the 1090 MHz receiver. | 0, 1 |
| subg | Enables or disables the Sub-GHz receiver. | 0, 1 |

### 3.14.2 Query Whether the Receiver(s) Are Enabled

```
AT+RX_ENABLE?
1090 Receiver: ENABLED
SubG Receiver: ENABLED
```

## 3.15 AT+RX_POSITION

Sets or queries the receiver position and the source used to determine it. A receiver position is required for some calculations, such as the distance to a tracked aircraft.

### 3.15.1 Query Receiver Position

```
AT+RX_POSITION?
Receiver Position:
    Source: FIXED [OK]
    Latitude: 37.000000 deg
    Longitude: -122.000000 deg
    GNSS Altitude: 0 ft
    Barometric Altitude: 0 ft
    Heading: 0.0 deg
    Speed: 0 kts
    ICAO: 0x000000 [NOT IN USE]
```

### 3.15.2 Set Receiver Position

```
AT+RX_POSITION=<source>,<args>
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| source | Source used to determine the receiver position. Each source takes different additional arguments, described below. | NONE, FIXED, GNSS, LOWEST, ICAO |

#### 3.15.2.1 Disable Receiver Position

```
AT+RX_POSITION=NONE
OK
```

#### 3.15.2.2 Set a Fixed Receiver Position

```
AT+RX_POSITION=FIXED,<lat_deg>,<lon_deg>,<gnss_alt_ft>,<baro_alt_ft>,<heading_deg>,<speed_kts>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| lat_deg | Latitude in degrees. | -90.0 to 90.0 |
| lon_deg | Longitude in degrees. | -180.0 to 180.0 |
| gnss_alt_ft | GNSS altitude in feet. | integer |
| baro_alt_ft | Barometric altitude in feet. | integer |
| heading_deg | Heading in degrees (true north). | 0.0 to 360.0 |
| speed_kts | Ground speed in knots. | >= 0 |

#### 3.15.2.3 Use the GNSS Receiver Position

```
AT+RX_POSITION=GNSS
OK
```

#### 3.15.2.4 Track the Lowest Aircraft

```
AT+RX_POSITION=LOWEST
OK
```

Bootstraps the receiver position from the lowest-altitude aircraft currently being tracked.

#### 3.15.2.5 Track an Aircraft by ICAO Address

```
AT+RX_POSITION=ICAO,<icao>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| icao | ICAO address (hex) of the aircraft to use for position bootstrap. | 0x000000 to 0xFFFFFF |

## 3.16 AT+RX_STATS

Reports OOK receiver statistics from the LR2021 radio: received packet count (pkt_rx), CRC and length error counts (crc_error, len_error), preamble detector fires (pbl_det), sync word successes and failures (sync_ok, sync_fail), and receive timeouts (timeout). Also reports receiver health counters: LR2021 RX FIFO overflows (fifo_full), raw packet queue overflows (q_ovf), single-bit errors corrected by the decoder (bitflips_fixed), and UAT receiver error restarts (uat_rx_restarts). The counters accumulate until reset.

```
AT+RX_STATS?
RX_STATS=pkt_rx=1234,crc_error=5,len_error=0,pbl_det=1500,sync_ok=1234,sync_fail=12,timeout=0,fifo_full=0,q_ovf=0,bitflips_fixed=42,uat_rx_restarts=0
```

Reset all statistics counters, including the health counters:

```
AT+RX_STATS=RESET
OK
```

## 3.17 AT+SETTINGS

Loads, saves, resets, or displays the device nonvolatile settings. Settings changed by other AT commands take effect immediately but are not retained across reboots until saved.

### 3.17.1 Query Current Settings

```
AT+SETTINGS?
```

Prints the current settings in a human-readable format.

### 3.17.2 Dump Settings in AT Format

```
AT+SETTINGS?DUMP
AT+PROTOCOL_OUT=CONSOLE,MAVLINK2
AT+R1090_GAIN=10
AT+R1090_PREAMBLE=MODE_S
AT+RX_ENABLE=,1,1
AT+SUBG_MODE=UAT_RX
...
```

Prints the current settings as AT commands that can be replayed to restore the configuration.

### 3.17.3 Load Settings from Nonvolatile Memory

```
AT+SETTINGS=LOAD
OK
```

### 3.17.4 Save Settings to Nonvolatile Memory

```
AT+SETTINGS=SAVE
OK
```

### 3.17.5 Reset Settings to Factory Defaults

```
AT+SETTINGS=RESET
OK
```

## 3.18 AT+SUBG_MODE

Selects the protocol mode of the Sub-GHz receiver. UAT_RX receives 978 MHz UAT ADS-B messages and ground uplink messages (FIS-B / TIS-B). UAT_RX_NO_UPLINK receives UAT ADS-B only: the uplink sync word is disabled at the RF core, so uplink frames are never received or decoded, which reduces receiver airtime and processor load when uplink data is not needed. Defaults to UAT_RX. Additional Sub-GHz protocols may be added as new modes in future firmware. Changes take effect immediately and persist across reboots after AT+SETTINGS=SAVE.

### 3.18.1 Set the Sub-GHz Receiver Mode

```
AT+SUBG_MODE=<mode>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| mode | Sub-GHz receiver protocol mode. | UAT_RX, UAT_RX_NO_UPLINK |

### 3.18.2 Query the Sub-GHz Receiver Mode

```
AT+SUBG_MODE?
SUBG_MODE=UAT_RX
```

## 3.19 AT+WATCHDOG

The watchdog timer automatically resets the device if firmware execution stalls and the timer is no longer serviced, providing automatic recovery from a hard fault. It can be disabled (for example, during breakpoint debugging) and can be self-tested via AT command to verify that it is still active.

### 3.19.1 Configure the Watchdog Timer

```
AT+WATCHDOG=<timeout_sec>
OK
```

| Parameter | Description | Acceptable Values |
|---|---|---|
| timeout_sec | Watchdog timeout in seconds. 0 disables the watchdog; 65535 is approximately 18.2 hours. | 0-65535 |

### 3.19.2 Query the Watchdog Timer

```
AT+WATCHDOG?
WATCHDOG=10
```

Prints the watchdog timeout in seconds, or 0 if the watchdog is disabled.

### 3.19.3 Test the Watchdog Timer

```
AT+WATCHDOG=TEST
Blocking for 11 seconds.
```

Blocks the processor for the configured timeout plus one second, which should induce a reboot. If no reboot occurs, an error indicates that the watchdog failed to activate.

# 4 Reporting Protocols

ADSBee m1421 supports the following reporting protocols on CONSOLE.

- CSBee
- GDL90
- MAVLINK 1
- MAVLINK 2
- Mode S Beast
- Raw Packets
- Aircraft JSON

The CONSOLE interface reports debug messages and AT command responses in addition to the selected reporting protocol. If the CONSOLE interface is being used as a reporting interface, it is recommended to send AT+LOG_LEVEL=SILENT to silence any debug logs that might corrupt the reported data, and to avoid sending additional AT commands while reading reported data in order to avoid the reported data being interspersed with OK and other AT command responses from the ADSBee m1421.

## 4.1 CSBee

Comma Separated Bee protocol containing information about tracked aircraft as plain text.

The CSBee protocol is heavily inspired by the Aerobits Aero CSV protocol.

### 4.1.1 CSBee Protocol Version History

Consult the table below for a version history of the CSBee protocol. Each version of the ADSBee firmware writes only the most recent version of the CSBee protocol. All versions of the CSBee protocol except for the most recent version are considered deprecated. Whenever possible, efforts are made to reduce breaking changes to the CSBee protocol when incrementing the version number.

Current protocol version: 3

| CSBee Protocol Version | Introductory Firmware Revision | Description |
|---|---|---|
| 1 | | Initial release. |
| 2 | adsbee_1090-0.9.0-rc1 | • Add UAT contacts.<br>• Change bit flag positions.<br>• Add dedicated fields for GNSS / baro vertical rates.<br>• Add additional aircraft info (e.g. dimensions). |
| 3 | adsbee_1090-0.9.0-rc15 | • Add is_non_transponder flag to Mode S aircraft.<br>• Prepend address qualifier to UAT ICAO addresses.<br>• Add is_utc_coupled flag to UAT aircraft. |

### 4.1.2 Mode S Aircraft Message

This message contains information about an aircraft being tracked via ADS-B (1090MHz). Aircraft reports are provided once per second, per aircraft, until contact with the aircraft has been lost for 60 seconds.

```
#A:ICAO,FLAGS,CALL,SQUAWK,ECAT,LAT,LON,BARO_ALT,GNSS_ALT,DIR,SPEED,BARO_VRATE,GNSS_VRATE,
NICNAC,ACDIMS,VERSION,SIGS,SIGQ,SFPS,ESFPS,CRC\r\n
```

| Field | Description | Format | Example value |
|---|---|---|---|
| #A | Mode S aircraft message start indicator. | | |
| ICAO | ICAO number of aircraft (3 bytes). | Hex Integer | 3C65AC |
| FLAGS | Flags bitfield, see table 4.1.2.1. | Hex Integer | 12F356A8 |
| CALL | Callsign of aircraft. | String | N61ZP |
| SQUAWK | SQUAWK of aircraft. | Octal Integer | 7232 |
| ECAT | Emitter category, see table 4.1.2.2. | Integer | 14 |
| LAT | Latitude, in degrees. | Float | 57.57634 |
| LON | Longitude, in degrees. | Float | 17.59554 |
| BARO_ALT | Barometric altitude, in feet. | Integer | 5000 |
| GNSS_ALT | Geometric altitude, in feet. | Integer | 5000 |
| DIR | Direction of aircraft, in degrees [0,360). Consult bit flags to determine whether this field is true heading, magnetic heading, or track. | Integer | 35 |
| SPEED | Horizontal velocity of aircraft, in knots. | Integer | 464 |
| BARO_VRATE | Barometric vertical velocity of aircraft, in ft/min. | Integer | -1344 |
| GNSS_VRATE | Geometric vertical velocity of the aircraft, in ft/min | Integer | -1344 |
| NICNAC | Aircraft data integrity. See table 4.1.2.4 | Hex Integer | |
| ACDIMS | Aircraft physical dimensions and system design assurance. See table 4.1.2.5. | Hex Integer | 31BE89F2 |
| VERSION | ADS-B protocol version used by the aircraft. | Integer | 2 |
| SIGS | Signal strength, in dBm. | Integer | -92 |
| SIGQ | Signal quality, in dB. | Integer | 2 |
| SFPS | Number of valid 56-bit Squitter Mode S frames received from the aircraft during the last second. | Integer | 2 |
| ESFPS | Number of valid 112-bit Extended Squitter Mode S frames received from the aircraft during the last second. | Integer | 5 |
| CRC | CRC16 (described in 4.1.2.6). | Hex Integer | 2D3E |

#### 4.1.2.1 FLAGS Bitfield

Note: All bits 25-31 are momentary (cleared and updated every reporting interval).

| Bit | Bit Name | Meaning if the bit is set (1) |
|---|---|---|
| 0 | IS_AIRBORNE | Emitter is airborne. |
| 1 | BARO_ALTITUDE_VALID | Emitter has provided a barometer altitude. |
| 2 | GNSS_ALTITUDE_VALID | Emitter has provided a GNSS altitude. |
| 3 | POSITION_VALID | Emitter has provided a pair of even and odd Compact Position Reporting packets that were decoded to a location. |
| 4 | DIRECTION_VALID | Emitter has provided a direction (may be ground track, magnetic compass heading, or true compass heading). |
| 5 | HORIZONTAL_SPEED_VALID | Emitter has provided a horizontal velocity. |
| 6 | BARO_VERTICAL_RATE_VALID | Emitter has provided a barometric vertical velocity. |
| 7 | GNSS_VERTICAL_RATE_VALID | Emitter has provided a GNSS vertical velocity. |
| 8 | IS_MILITARY | Emitter has transmitted at least one packet using a military format, such as Military Extended Squitter (DF=19). |
| 9 | IS_CLASS_B2_GROUND_VEHICLE | Emitter is actually a ground vehicle using a Class B2 transponder with a transmission power < 70W. |
| 10 | HAS_1090_ES_IN | Emitter has receive capability for 1090MHz Extended Squitter transmissions. |
| 11 | HAS_UAT_IN | Emitter has receive capability for UAT (978MHz Universal Access Transceiver) transmissions. |
| 12 | TCAS_OPERATIONAL | Emitter has a functional TCAS (Traffic Collision Avoidance System) onboard. |
| 13 | SINGLE_ANTENNA | Emitter is using a single antenna, instead antennas above and below the fuselage. Transmissions may be weak or irregular during maneuvering. |
| 14 | DIRECTION_IS_HEADING | Surface position messages provided by the aircraft indicate a heading and not a track angle. |
| 15 | HEADING_USES_MAGNETIC_NORTH | Heading reported by the aircraft while on the surface uses magnetic north instead of true north. |
| 16 | IDENT | The aircraft has its SPI (Special Position Identification) bits set in Mode A/C or Mode S messages. This indicates that the pilot has depressed the momentary IDENT switch on their transponder, most likely at the request of air traffic control. |
| 17 | ALERT | The aircraft is issuing either a permanent or momentary alert. This could correspond to an operational mode change or something else. |
| 18 | TCAS_RA | The aircraft has an active TCAS resolution advisory (i.e. the aircraft is warning the pilot to take action in order to avoid colliding with another aircraft). |
| 19 | IS_NON_TRANSPONDER | This aircraft is being rebroadcast (not received from a transponder). |
| 20-22 Reserved | | |
| 23 | UPDATED_BARO_ALTITUDE | Barometric altitude has been updated within the last reporting interval. |
| 24 | UPDATED_GNSS_ALTITUDE | GNSS altitude has been updated within the last reporting interval. |
| 25 | UPDATED_POSITION | Position (latitude / longitude) has been updated within the last reporting interval. |
| 26 | UPDATED_DIRECTION | Direction has been updated within the last reporting interval. |
| 27 | UPDATED_HORIZONTAL_SPEED | Horizontal velocity has been updated within the last reporting interval. |
| 28 | UPDATED_BARO_VERTICAL_RATE | Barometric vertical velocity has been updated within the last reporting interval. |
| 29 | UPDATED_GNSS_VERTICAL_RATE | GNSS vertical velocity has been updated within the last reporting interval. |
| 30-31 Reserved | | |

#### 4.1.2.2 ECAT Field

The ECAT field indicates the Emitter Category (i.e. airframe type) for each ADSB emitter that is being tracked. This field contains information about what kind of aircraft, ground vehicle, obstacle, or other airspace user is emitting ADS-B packets, and can be used to understand the emitter's maneuvering capability and potential for wake vortex impact.

| ECAT Value | Emitter Category |
|---|---|
| -1 | Invalid |
| 0 | No Category Information |
| 1 | Light Aircraft (< 7,000kg) |
| 2 | Medium 1 (7,000kg – 34,000kg) |
| 3 | Medium 2 (34,000kg – 136,000kg) |
| 4 | High Vortex Aircraft |
| 5 | Heavy (> 136,000kg) |
| 6 | High Performance (> 5 G acceleration and > 400 kts speed) |
| 7 | Rotorcraft |
| 8 | Reserved |
| 9 | Glider / Sailplane |
| 10 | Lighter Than Air |
| 11 | Parachutist / Skydiver |
| 12 | Ultralight / Hang Glider / Paraglider |
| 13 | Reserved 1 |
| 14 | Unmanned Aerial Vehicle |
| 15 | Space / Transatmospheric Vehicle |
| 16 | Reserved 2 |
| 17 | Surface Emergency Vehicle |
| 18 | Surface Service Vehicle |
| 19 | Point Obstacle |
| 20 | Cluster Obstacle |
| 21 | Line Obstacle |

#### 4.1.2.3

#### 4.1.2.4 NICNAC Bitfield

| NICNAC Bitfield | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| Source Integrity Level (SIL) | | Geometric Vertical Accuracy (GVA) | | Navigation Accuracy Category: Position ($NAC_p$) | | | | Navigation Accuracy Category: Velocity ($NAC_v$) | | | Navigation Integrity Category: Barometer ($NIC_{baro}$) | Navigation Integrity Category (NIC) | | | |

**NICNAC[0-3]: Navigation Integrity Category (NIC)**

The radius of containment (NIC) indicates how much trust should be placed in an aircraft's reported location in the horizonal plane. The NIC reports a radius of containment specified by the avionics system of the emitter. The probability that the aircraft is outside of this radius of containment due to its avionics system receiving a faulty signal from one of its inputs (without displaying an error) is provided by another bitfield called the Source Integrity Level (SIL). Combined, the NIC and SIL indicate how likely it is that an aircraft is not actually contained by a bubble of a specified size, centered at

the aircraft's reported location, assuming that the avionics onboard the aircraft are functioning correctly but may be given faulty inputs.

A higher NIC value indicates more trust in an aircraft's reported latitude / longitude position.

| **NIC Value** | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Radius of Containment** | Unknown | < 20 NM | < 8 NM | < 4 NM | < 2 NM | < 1 NM | < 0.6 NM | < 0.2 NM | < 0.1 NM | < 75 m | < 25 m | < 7.5 m |

**NICNAC [4]: Navigation Integrity Category: Barometer ($NIC_{baro}$)**

The barometric altitude integrity ($NIC_{baro}$) indicates how much trust should be placed in an aircraft's reported altitude. The field is a single bit that indicates whether the aircraft uses an altimeter that has been cross-checked against other sources. Old school encoding altimeters have many parallel wires and output an altitude in a format called a Gillham Code, and have no built-in method of error checking. A single faulty wire can result in erroneous readings, so this bit lets air traffic control know whether to take altitude readings from the aircraft with a grain of salt. A 0 indicates that the transponder is outputting altitude from a Gillham coded source (with no way to cross check the value), while a 1 indicates that the transponder is outputting altitude from a Gillham coded source while using another sensor to cross-check it, or is using a more modern barometer that supports a protocol with built-in error checking.

A higher $NIC_{baro}$ value (i.e. 1 instead of 0) indicates more trust in the aircraft's reported altitude.

| **$NIC_{baro}$ Value** | 0 | 1 |
|---|---|---|
| **Barometric Altitude Integrity** | Altitude is from a Gillham-coded input, not cross checked. | Altitude is from a Gillham-coded input that is being cross-checked with another source, or from a non Gillham-coded input with built-in error checking features. |

**NICNAC [5-7]: Navigation Accuracy Category: Velocity ($NAC_v$)**

The horizontal velocity error ($NAC_v$) indicates the expected accuracy of the reported velocity of the aircraft when systems are operating nominally. This varies depending on the accuracy capabilities of the measurement equipment onboard the aircraft, and not how often we expect said equipment to fail.

A higher $NAC_v$ indicates a more accurate velocity measurement system.

| **$NAC_v$ Value** | **Horizontal Velocity Error** |
|---|---|
| 0b000 | Unknown or ≥ 10 m/s |
| 0b110 | < 10 m/s |
| 0b010 | < 3 m/s |
| 0b011 | < 1 m/s |
| 0b100 | < 0.3 m/s |

**NICNAC [8-11]: Navigation Accuracy Category: Position ($NAC_p$)**

The estimated position uncertainty ($NAC_p$) indicates the expected accuracy of the aircraft's reported location when systems are operating nominally. This varies depending on the capabilities of the aircraft's positioning system and not on how often we expect said equipment to fail.

A higher $NAC_p$ indicates a more accurate positioning system.

| **$NAC_p$ Value** | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Estimated Position Uncertainty** | Unknown or ≥ 10NM | < 10 NM | < 4 NM | < 2 NM | < 1 NM | < 0.5 NM | < 0.3 NM | < 0.1 NM | < 0.5 NM | < 30 m | < 10 m | < 3 m |

**NICNAC[12-13]: Geometric Vertical Accuracy (GVA)**

The geometric vertical accuracy indicates the 95% confidence interval (vertical figure of merit) provided by the aircraft's onboard GNSS system (i.e. assuming some distribution of altitudes, the aircraft's GNSS system is confident that 95 out of 100 times, the aircraft falls within some height of the reported geometric altitude.

| **GVA Value** | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| **95% Vertical Figure of Merit (VFOM)** | Unknown or ≥ 150 m | < 150 m | ≤ 45 m | < 45 m (Was previously "reserved", the actual value of this field may change but is guaranteed to be < 45 m). |

**NICNAC[14-15]: Source Integrity Level (SIL)**

The source integrity level indicates the probability that the aircraft exceeds the bounds of its horizontal radius of containment (NIC) due to a silent fault in signals received by the aircraft (no avionics failure).

| **SIL Value** | **Probability of Exceeding NIC Radius of Containment Due to Silent Fault** |
|---|---|
| 0 | Unknown or > $1x10^{-3}$ per flight hour. |
| 1 | ≤ $1x10^{-3}$ per flight hour. |
| 2 | ≤ $1x10^{-5}$ per flight hour. |
| 3 | ≤ $1x10^{-7}$ per flight hour. |
| 4 | Unknown or > $1x10^{-3}$ per sample. |
| 5 | ≤ $1x10^{-3}$ per sample. |
| 6 | ≤ $1x10^{-5}$ per sample. |
| 7 | ≤ $1x10^{-7}$ per sample. |

#### 4.1.2.5 ACDIMS Bitfield

| **ACDIMS Bitfield** | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| GNSS Offset Forward of Reference Point (GOF) | | | | | | | | GNSS Offset Right of Reference Point (GOR) | | | | | | | |

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Aircraft Length (ACL) | | | | | | | Aircraft Width (ACW) | | | | | | | System Design Assurance (SDA) | |

### ACDIMS[0-1]: System Design Assurance (SDA)

The system design assurance indicates how robust the aircraft's position reporting systems are to failures of various severities. For instance, SDA = 1, a low SDA value, corresponds to Software and Hardware Design Assurance Level D, which states that a minor failure could cause the aircraft to transmit misleading position information with a probability of ≤ $1x10^{-3}$ per flight hour. A more robust system with SDA = 3, corresponding to Software and Hardware Design Assurance Level B, is expected to transmit misleading position information with a probability of $1x10^{-7}$ per flight hour even under a Hazardous failure condition.

Software Design Assurance categories used in this field are classified under RTCA DO-178B, Airborne Electronic Hardware Design Assurance are classified under RTCA DO-254, and failure classification levels are defined in FAA Advisory Circular AC-23.1309-1E.

| SDA Value | Supported Failure Condition | Probability of Undetected Fault causing transmission of False or Misleading Information | Software and Hardware Design Assurance Level |
|---|---|---|---|
| 0 | Unknown / No Safety Effect | > $1x10^{-3}$ per flight hour or unknown. | N/A |
| 1 | Minor | ≤ $1x10^{-3}$ per flight hour. | D |
| 2 | Major | ≤ $1x10^{-5}$ per flight hour. | C |
| 3 | Hazardous | ≤ $1x10^{-7}$ per flight hour. | B |

### ACDIMS[2-8] Aircraft Width (ACW)

Approximate value for the width of the aircraft bounding box, in meters. Can be interpreted as a 7-bit unsigned integer.

### ACDIMS[9-15] Aircraft Length (ACL)

Approximate value for the length of the aircraft bounding box, in meters. Can be interpreted as a 7-bit unsigned integer.

### ACDIMS[16-23] GNSS Offset Right of Reference Point (GOR)

Signed 8-bit integer representing the left/right offset of the GNSS antenna from the aircraft's reference point. A positive value indicates that the GNSS antenna is to the right of the aircraft reference point.

### ACDIMS[24-31] GNSS Offset Forward of Reference Point (GOF)

Signed 8-bit integer representing the forward/back offset of the GNSS antenna from the aircraft's reference point. A positive value indicates that the GNSS antenna is forward of the aircraft's reference point. Always 0 for Mode S aircraft.

#### 4.1.2.6 CRC Field

CSBee messages use a 16-bit Cyclical Redundancy Checksum (CRC-16), which can be calculated using the algorithm in the C++ code snippet below. Note the “swap16” helper function which also needs to be included.

```
uint16_t swap16(uint16_t value) { return (value << 8) | (value >> 8); }

uint16_t CalculateCRC16(const uint8_t *data_p, int32_t length) {
    uint8_t x;
    uint16_t crc = 0xFFFF;
    while (length--) {
        x = crc >> 8 ^ *data_p++;
        x ^= x >> 4;
        crc = (crc << 8) ^ ((uint16_t)(x << 12)) ^ ((uint16_t)(x << 5)) ^ ((uint16_t)x);
    }
    return swap16(crc);
}
```

### 4.1.3 UAT Aircraft Message

```
#U:ICAO,UAT_FLAGS,CALL,SQUAWK,ECAT,LAT,LON,BARO_ALT,GNSS_ALT,DIR,SPEED,BARO_VRATE,GNSS_VR
ATE,UAT_EMERG,NICNAC,ACDIMS,VERSION,SIGS,SIGQ,UATFPS,CRC\r\n
```

| Field | Description | Format | Example value |
|---|---|---|---|
| #U | UAT aircraft message start indicator. | | |
| ICAO | First Digit (optional): Address Qualifier (1 digit: see table 4.1.3.1)<br>Last 6 Digits: ICAO address of aircraft (6 hex digits). | Hex Integer | 13C65AC<br>3C65AC |
| UAT_FLAGS | Flags bitfield, see table 4.1.3.2. | Hex Integer | 12F356A8 |
| CALL | Callsign of aircraft. | String | N61ZP |
| SQUAWK | SQUAWK of aircraft. | Octal Integer | 7232 |
| ECAT | Emitter category, see table 4.1.2.2. | Integer | 14 |
| LAT | Latitude, in degrees. | Float | 57.57634 |
| LON | Longitude, in degrees. | Float | 17.59554 |
| BARO_ALT | Barometric altitude, in feet. | Integer | 5000 |
| GNSS_ALT | Geometric altitude, in feet. | Integer | 5000 |
| DIR | Direction of aircraft, in degrees [0,360). Consult bit flags to determine whether this field is true heading, magnetic heading, or track. | Integer | 35 |
| SPEED | Horizontal velocity of aircraft, in knots. | Integer | 464 |
| BARO_VRATE | Barometric vertical velocity of aircraft, in ft/min. | Integer | -1344 |
| GNSS_VRATE | Geometric vertical velocity of the aircraft, in ft/min | Integer | -1344 |
| UAT_EMERG | Aircraft UAT emergency / priority status. See table. | Integer | 0 |
| NICNAC | Aircraft data integrity. See table 4.1.2.4 | Hex Integer | |
| ACDIMS | Aircraft physical dimensions and system design assurance. See table 4.1.2.5. | Hex Integer | 31BE89F2 |
| VERSION | UAT protocol version used by the aircraft. | Integer | 1 |
| SIGS | Signal strength, in dBm. | Integer | -92 |
| SIGQ | Number of bits corrected using UAT forward error correction. 0 (best), 6 (worst). | Integer | 2 |
| UATFPS | Number of valid UAT frames received from the aircraft in the last second. | Integer | 1 |
| CRC | CRC16 (described in 4.1.2.6). | Hex Integer | 2D3E |

NOTE: See Mode S Aircraft message decoding reference for any bitfields not prefixed with "UAT_".

#### 4.1.3.1 UAT Address Qualifier

| Address Qualifier Value | Meaning |
|---|---|
| 0 | Contact is ADSB target (broadcast by transponder) with 24-bit ICAO address. |
| 1 | Contact is ADSB target (broadcast by transponder) with self-assigned temporary address. |
| 2 | Contact is TIS-B target (broadcast by ground station), using a 24-bit ICAO address. |
| 3 | Contact is TIS-B target (broadcast by ground stations) with track file identifier. |
| 4 | Contact is a surface vehicle. |
| 5 | Contact is a fixed ADSB beacon. |

#### 4.1.3.2 UAT_FLAGS Bitfield

| Bit | Bit Name | Meaning if the bit is set (1) |
|---|---|---|
| 0 | IS_AIRBORNE | Emitter is airborne. |
| 1 | BARO_ALTITUDE_VALID | Emitter has provided a barometer altitude. |
| 2 | GNSS_ALTITUDE_VALID | Emitter has provided a GNSS altitude. |
| 3 | POSITION_VALID | Emitter has provided a pair of even and odd Compact Position Reporting packets that were decoded to a location. |
| 4 | DIRECTION_VALID | Emitter has provided a direction (may be ground track, magnetic compass heading, or true compass heading). |
| 5 | HORIZONTAL_SPEED_VALID | Emitter has provided a horizontal velocity. |
| 6 | BARO_VERTICAL_RATE_VALID | Emitter has provided a barometric vertical velocity. |
| 7 | GNSS_VERTICAL_RATE_VALID | Emitter has provided a GNSS vertical velocity. |
| 8 | HAS_CDTI | Aircraft has Cockpit Display of Traffic Information (CDTI) equipped. |
| 9 | TCAS_OPERATIONAL | Aircraft is equipped with an operational TCAS / ACAS system. |
| 10 | DIRECTION_IS_HEADING | Surface position messages provided by the aircraft indicate a heading and not a track angle. |
| 11 | HEADING_USES_MAGNETIC_NORTH | Heading reported by the aircraft while on the surface uses magnetic north instead of true north. |
| 12 | IDENT | The aircraft has its SPI (Special Position Identification) bits set in Mode A/C or Mode S messages. This indicates that the pilot has depressed the momentary IDENT switch on their transponder, most likely at the request of air traffic control. |
| 13 | IS_UTC_COUPLED | The aircraft is transmitting frames precisely aligned to the UTC second. |
| 14 Reserved | | |
| 15 | TCAS_RA | The aircraft has an active TCAS resolution advisory (i.e. the aircraft is warning the pilot to take action in order to avoid colliding with another aircraft). |
| 16 | RECEIVING_ATC_SERVICES | |
| 17-22 Reserved | | |
| 23 | UPDATED_BARO_ALTITUDE | Barometric altitude has been updated within the last reporting interval. |
| 24 | UPDATED_GNSS_ALTITUDE | GNSS altitude has been updated within the last reporting interval. |
| 25 | UPDATED_POSITION | Position (latitude / longitude) has been updated within the last reporting interval. |
| 26 | UPDATED_DIRECTION | Direction has been updated within the last reporting interval. |
| 27 | UPDATED_HORIZONTAL_SPEED | Horizontal velocity has been updated within the last reporting interval. |
| 28 | UPDATED_BARO_VERTICAL_RATE | Barometric vertical velocity has been updated within the last reporting interval. |
| 29 | UPDATED_GNSS_VERTICAL_RATE | GNSS vertical velocity has been updated within the last reporting interval. |
| 30-31 Reserved | | |

#### 4.1.3.3 UAT_EMERG Bitfield

Contains the emergency / priority status reported by a UAT emitter.

| UAT_EMERG Value | Meaning |
|---|---|
| 0 | No emergency. |
| 1 | General emergency. |
| 2 | Lifeguard / medical emergency. |
| 3 | Minimum fuel. |
| 4 | No communications. |
| 5 | Unlawful interference. |
| 6 | Downed aircraft. |
| 7 | Reserved. |

### 4.1.4 Statistics Message

This message contains some useful statistics about ADSBee's operational status. Format of that frame is shown below:

```
#S:VERSION,DPS,RAW_SFPS,SFPS,RAW_ESFPS,ESFPS,RAW_UATFPS,UATFPS,TSCAL,UPTIME,CRC\r\n
```

| Field | Description | Format | Example Value |
|---|---|---|---|
| #S | Statistics message start indicator. | | |
| VERSION | Version of the CSBee protocol being used. | Integer | 3 |
| SDPS | Number of attempted Mode S demodulations in the last second. | Integer | 106 |
| RAW_SFPS | Number of squitter (56-bit Mode S) frames received in the last second. | Integer | 150 |
| SFPS | Number of valid squitter (56-bit Mode S) frames received in the last second. | Integer | 20 |
| RAW_ESFPS | Number of extended squitter (112-bit Mode S) frames received in the last second. | Integer | 15 |
| ESFPS | Number of valid Mode S frames received in the last second. | Integer | 3 |
| RAW_UATFPS | Number of UAT frames received in the last second. | Integer | 5 |
| UATFPS | Number of valid UAT frames received in the last second. | Integer | 4 |
| NUM_AIRCRAFT | Number of aircraft tracked in the last second. | Integer | 20 |
| TSCAL | Calibration value for TS field in raw frames | Integer | 13999415 |
| UPTIME | Receiver uptime, in seconds. | Integer | 13456 |
| CRC | CRC16 (described in 4.1.4.1). | Hex Integer | 2D3E |

#### 4.1.4.1 CRC Field

See 4.1.2.6.

## 4.2 GDL90

ADSBee complies with the GDL90 public data interface specification 560-1058-00 Rev A. The GDL90_NO_UAT_UPLINK protocol variant is identical to GDL90, except that UAT uplink messages are not forwarded.

## 4.3 MAVLINK

Tracked aircraft information is sent in MAVLINK ADSB_VEHICLE messages, in a data burst once per second. The data burst consists of 1x MAVLINK Heartbeat message, Nx ADSB_VEHICLE messages, where N is the number of tracked aircraft, and 1x special delimiter message (MAVLINK_REQUEST_DATA_STREAM for MAVLINK1 mode, MAVLINK_MESSAGE_INTERVAL for MAVLINK2 mode) which indicates the end of the list of tracked aircraft. Note that this is a binary protocol which is not human readable.

**WARNING**: Windows has a bug, which causes some machines to recognize a serial port reporting MAVLINK packets as a mouse, which can result in phantom mouse movements and clicks (ask me how I know). Please use caution while running MAVLINK on a serial port while the computer is unattended.

### 4.3.1 MAVLINK_HEARTBEAT ( Message ID 0 ) Packet Definition

| Field Name | Type | Units | Values | Description |
|---|---|---|---|---|
| custom_mode | `uint32_t` | | 0 | Unused. |
| type | `uint8_t` | | MAV_TYPE_ADSB | Always MAV_TYPE_ADSB (27). |
| autopilot | `uint8_t` | | MAV_AUTOPILOT_INVALID | Always MAV_AUTOPILOT_INVALID (8). |
| base_mode | `uint8_t` | | 0 | Unused. |
| system_status | `uint8_t` | | MAV_STATE_ACTIVE | Always MAV_STATE_ACTIVE (4). |
| mavlink_version | `uint8_t` | | MAVLINK_VERSION | 1 for MAVLINK1 mode, 2 for MAVLINK2 mode. |

### 4.3.2 MAVLINK ADSB_VEHICLE (Message ID 246) Packet Definition

From: https://mavlink.io/en/messages/common.html#ADSB_VEHICLE

| Field Name | Type | Units | Values | Description |
|---|---|---|---|---|
| ICAO_address | `uint32_t` | | | ICAO address |
| lat | `int32_t` | degE7 | | Latitude |
| lon | `int32_t` | degE7 | | Longitude |
| altitude_type | `uint8_t` | | ADSB_ALTITUDE_TYPE | ADSB altitude type. |
| altitude | `int32_t` | mm | | Altitude(ASL) |
| heading | `uint16_t` | cdeg | | Course over ground |
| hor_velocity | `uint16_t` | cm/s | | The horizontal velocity |
| ver_velocity | `int16_t` | cm/s | | The vertical velocity. Positive is up |
| callsign | `char[9]` | | | The callsign, 8+null |
| emitter_type | `uint8_t` | | ADSB_EMITTER_TYPE | ADSB emitter type. |
| tslc | `uint8_t` | s | | Time since last communication in seconds |
| flags | `uint16_t` | | ADSB_FLAGS | Bitmap to indicate various statuses including valid data fields |
| squawk | `uint16_t` | | | Squawk code |

### 4.3.3 MAVLINK_REQUEST_DATA_STREAM (Message ID 66) Packet Definition

This message serves as the delimiter which indicates the end of the aircraft list in MAVLINK1 mode.

| Field Name | Type | Units | Description |
|---|---|---|---|
| req_message_rate | `uint16_t` | | Always 0 (unused). |
| target_system | `uint8_t` | | Always 0 (unused). |
| target_component | `uint8_t` | | Always 0 (unused). |
| req_stream_id | `uint8_t` | | Always 0 (unused). |
| start_stop | `uint8_t` | | Always 0 (unused). |

### 4.3.4 MAVLINK MESSAGE_INTERVAL (Message ID 244) Packet Definition

This message serves as the delimiter which indicates the end of the aircraft list in MAVLINK2 mode.

From: https://mavlink.io/en/messages/common.html#MESSAGE_INTERVAL

| Field Name | Type | Units | Description |
|---|---|---|---|
| message_id | `uint16_t` | | The ID of the requested MAVLink message. v1.0 is limited to 254 messages.<br><br>NOTE: For ADSBee m1421, message_id is always 246, corresponding to the ADSB_VEHICLE message. |
| interval_us | `int32_t` | us | The interval between two messages. A value of -1 indicates this stream is disabled, 0 indicates it is not available, > 0 indicates the interval at which it is sent.<br><br>NOTE: For ADSBee m1421, interval_us is always 1000 us. |

## 4.4 Mode S Beast

Mode S Beast is a popular feed protocol used by both open-source and commercial data exchanges to send Mode S data, often over TCP sockets or serial connections.

Mode S Beast sends packets as raw binary, with 0x1a used as an "escape" character indicating the start of a Beast frame. Any non start-of-frame instance of 0x1a is escaped with an additional 0x1a beforehand. Thus, 0x1a can be interpreted as the start of a Beast frame, and 0x1a1a can be interpreted as a 0x1a value within a Beast frame.

### 4.4.1 Beast: Mode S Frames

| Field | Frame Start | Frame Type | Timestamp | RSSI | Payload |
|---|---|---|---|---|---|
| Length [Bytes] | 1 | 1 | 6-12 | 1-2 | Mode S Short: 7-14<br>Mode S Long: 14-28 |
| Example | 0x1a | 0x32 | 0x123456781a1aff | 0x56 | <data> |
| Definition | 0x1a<br><br>Frame start is always an escape char. | 0x32: Mode S Short<br>0x33: Mode S Long<br><br>No escapes. | 6-byte 12MHz MLAT timestamp, MSB first.<br><br>May have up to 6 escapes. | RSSI byte is "Beast Power Level", which is the square root of the un-logged dBFS value.<br><br>May have up to 1 escape. | Packet contents.<br><br>May theoretically have up to <packet_length> escapes, but this would not correspond to a valid packet and will not be seen in practice. |

### 4.4.2 Beast: UAT Frames

| Field | Frame Start | Frame Type | UAT Message Type | Timestamp | RSSI | Payload |
|---|---|---|---|---|---|---|
| Length [Bytes] | 1 | 1 | 1 | 6-12 | 1-2 | UAT Short ADSB: |
| Example | 0x1a | 0xec | 0x73 | 0x123456781a1aff | 0x56 | <data> |
| Definition | 0x1a<br><br>Frame start is always an escape char. | 0xec<br><br>Indicates frame type UAT (both ADSB and uplink).<br><br>No escapes. | 0x73 ('s'): UAT short ADSB (30 byte payload, not including escapes).<br>0x5c ('l'): UAT long ADSB (48 byte payload, not including escapes)<br>0x75 ('u'): UAT Uplink (552 byte payload, not including escapes)<br><br>No escapes. | 6-byte 12MHz MLAT timestamp, MSB first.<br><br>May have up to 6 escapes. | RSSI byte is "Beast Power Level", which is the square root of the un-logged dBFS value.<br><br>May have up to 1 escape. | Packet contents, including both payload and FEC parity.<br><br>Uplink packets remain interleaved (transmitted byte order retained).<br><br>May theoretically have up to <packet_length> escapes, but this would not correspond to a valid packet and will not be seen in practice. |

## 4.5 Raw Packets

### 4.5.1 Raw Mode S Frames

Raw Mode S packets are reported in the format below. Only packets that have passed checksum validation and match a supported Mode S downlink format are reported.

```
#MDS*RAW_FRAME;(SOURCE,SIGS,SIGQ,TS)\r\n
```

| Field | Description | Format | Example value |
|---|---|---|---|
| #MDS | Indicates the start of a raw Mode S frame. | | |
| RAW_FRAME | The packet contents as raw hexadecimal. Varies in length depending on packet length. | Hexadecimal String | 8D48C22D60AB0452BFAD19A695E0 |
| SOURCE | An unsigned integer | `int16_t` | 2 |
| SIGS | Signal strength of the packet in dBm, printed as a decimal value. | `int16_t` | -60 |
| SIGQ | Signal quality of the packet in dB, printed as a decimal value. | `int16_t` | 2 |
| TS | 64-bit 48MHz MLAT counter ticks, printed as a hexadecimal string. | `uint64_t` | 00000000FB671342 |

### 4.5.2 Raw UAT ADSB Frames

Raw UAT frames are reported in the format below. Only packets that have passed checksum validation are reported. Note that UAT uplink frames can be quite long (552 Bytes / 1104 chars), so serial buffers need to be able to handle this!

```
#UAT*-RAW_FRAME;(SIGS,SIGQ,TS)\r\n
```

| Field | Description | Format | Example value |
|---|---|---|---|
| #UAT | Indicates the start of a raw UAT frame. | | |
| RAW_FRAME | The packet contents as raw hexadecimal. Varies in length depending on packet length. | Hexadecimal String | |
| SIGS | Signal strength of the packet in dBm. | `int16_t` | -60 |
| SIGQ | Signal quality of the packet in dB. | `int16_t` | 2 |
| TS | 64-bit MLAT counter value in hexadecimal format. | `uint64_t` | 00000000FB671342 |

### 4.5.3 Raw UAT Uplink Frames

Raw UAT uplink frames are reported in the format below. Only packets that have passed checksum validation are reported. Note that UAT uplink frames can be quite long (552 Bytes / 1104 chars), so serial buffers need to be able to handle this!

```
#UAT*+RAW_FRAME;(SIGS,SIGQ,TS)\r\n
```

| Field | Description | Format | Example value |
|---|---|---|---|
| #UAT | Indicates the start of a raw UAT frame. | | |
| RAW_FRAME | The packet contents as raw hexadecimal. Varies in length depending on packet length. | Hexadecimal String | |
| SIGS | Signal strength of the packet in dBm. | `int16_t` | -60 |
| SIGQ | Signal quality of the packet in dB. | `int16_t` | 2 |
| TS | 64-bit MLAT counter value in hexadecimal format. | `uint64_t` | 00000000FB671342 |

## 4.6 Aircraft JSON

The ADSBee m1421 streams real-time aircraft data over the Console UART when its reporting protocol is set to AIRCRAFT_JSON (AT+PROTOCOL_OUT=CONSOLE,AIRCRAFT_JSON). The full aircraft list is sent once per second as one JSON object per tracked aircraft, with objects separated by newline characters.

The ADSBee Aircraft JSON format is intended to be compatible with the JSON format emitted by readsb (https://github.com/wiedehopf/readsb/blob/dev/README-json.md).

### 4.6.1 Message Format

The Console UART stream contains one or more newline-terminated JSON objects, one per tracked aircraft. Fields are omitted when their data has not been received or is not applicable to the aircraft type; clients must treat absent fields as unknown rather than zero.

```
{"hex":"a1b2c3","type":"adsb_icao","flight":"UAL123
","lat":37.62345,"lon":-122.37812,"alt_baro":37000,"gs":480,"track":270.0,"baro_rate":0,"
nic":8,"nic_baro":1,"nac_p":9,"nac_v":2,"sil":3,"gva":2,"sda":2,"version":2,"rssi":-68,"m
essages":142}
{"hex":"c3d4e5","type":"adsb_icao","on_ground":1,"lat":37.61200,"lon":-122.38500,"alt_bar
o":0,"gs":12,"track":90.0,"nic":8,"nic_baro":1,"nac_p":9,"nac_v":2,"sil":3,"gva":2,"sda":
2,"version":2,"rssi":-71,"messages":23}
```

### 4.6.2 JSON Fields

| Field | Type | Condition | Mode S | UAT | Units | Description |
|---|---|---|---|---|---|---|
| hex | string | Always | ✓ | ✓ | — | 6-digit ICAO 24-bit address in lowercase hex. UAT non-ICAO addresses (self-assigned or TIS-B track file) are prefixed with ~. |
| type | string | Always | ✓ | ✓ | — | Aircraft type. Mode S: adsb_icao, adsr_icao (non-transponder), adsb_icao_nt (Class B ground vehicle). UAT: adsb_icao, adsb_other, tisb_icao, tisb_trackfile. |
| flight | string | If callsign received | ✓ | ✓ | — | 8-character flight identifier / callsign, right-padded with spaces. |
| squawk | string | If squawk received | ✓ | ✓ | — | 4-digit octal transponder squawk code. |
| category | string | If category received | ✓ | ✓ | — | Emitter category in readsb A0-D7 format. Letter A-D = set 0-3; digit 0-7 = subcategory. |
| lat | number | If position valid | ✓ | ✓ | degrees | WGS-84 latitude, 5 decimal places (~1.1 m resolution). |
| lon | number | If position valid | ✓ | ✓ | degrees | WGS-84 longitude, 5 decimal places (~1.1 m resolution). |
| r_dst | number | If distance known | ✓ | ✓ | NM | Distance from the receiver position (see AT+RX_POSITION), 3 decimal places. Within 0.015% + 1 m of the WGS-84 geodesic distance up to 600 km. |
| r_dir | number | If distance known | ✓ | ✓ | degrees | Bearing from the receiver position, clockwise from true north, 1 decimal place. Within 0.2 degrees of the WGS-84 geodesic bearing. |
| alt_baro | integer | If baro altitude valid | ✓ | ✓ | ft | Barometric pressure altitude. |
| alt_geom | integer | If GNSS altitude valid | ✓ | ✓ | ft | Geometric (GNSS/GPS) altitude. |
| gs | integer | If speed valid | ✓ | ✓ | kt | Ground speed. |
| track | number | If direction valid, not heading | ✓ | ✓ | degrees | True track over ground, 1 decimal place. |
| mag_heading | number | If heading valid, magnetic | ✓ | ✓ | degrees | Magnetic heading, 1 decimal place. |
| true_heading | number | If heading | ✓ | ✓ | degrees | True heading, 1 decimal place. |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | valid, true north | | | | |
| baro_rate | integer | If baro vertical rate valid | ✓ | ✓ | ft/min | Barometric vertical rate of climb/descent. |

| | | | | | | |
|---|---|---|---|---|---|---|
| geom_rate | integer | If GNSS vertical rate valid | ✓ | ✓ | ft/min | Geometric (GNSS) vertical rate of climb/descent. |
| nic | integer | Always | ✓ | ✓ | — | Navigation Integrity Category (0-11). |
| nic_baro | integer | Always | ✓ | ✓ | — | Navigation Integrity Category Barometric (0-4). |
| nac_p | integer | Always | ✓ | ✓ | — | Navigation Accuracy Category - Position (0-11). |
| nac_v | integer | Always | ✓ | ✓ | — | Navigation Accuracy Category - Velocity (0-4). |
| sil | integer | Always | ✓ | ✓ | — | Surveillance Integrity Level (0-3). |
| gva | integer | Always | ✓ | ✓ | — | Geometric Vertical Accuracy (0-2). |
| sda | integer | Always | ✓ | ✓ | — | System Design Assurance (0-3). |
| version | integer | Always (Mode S); if >= 0 (UAT) | ✓ | ✓ | — | ADS-B version number. |
| rssi | integer | Always | ✓ | ✓ | dBm | Last received signal strength. |
| messages | integer | Always | ✓ | ✓ | — | Total valid messages received from this aircraft since tracking began. |
| alert | integer (1) | If alert flag set | ✓ | ✗ | — | Present (value 1) when the Mode S alert flag is active. Mode S only. |
| spi | integer (1) | If SPI flag set | ✓ | ✗ | — | Present (value 1) when the aircraft is squawking ident (Special Position Identification). Mode S only. |
| emergency | string | If emergency status set | ✗ | ✓ | — | Emergency/priority status: none, general, lifeguard, minfuel, nordo, unlawful, downed, reserved. UAT only. |
| on_ground | integer (1) | If NOT airborne | ✓ | ✓ | — | Present (value 1) when aircraft is on the ground. For UAT, also set for surface vehicles. |